D1 – Design Specification (Part 2)

Recipix v4.1 – a domain-specific language
for parameterized recipes with dimensional types

Berk Hakan Öge (210104004132) İsmail Kabak (1901042652)

22 May 2026

1. Language Overview [P1, revised]

Recipix is a domain-specific language for describing, parameterizing, scaling, and substituting recipes. Every ingredient quantity carries a dimension (Mass, Volume, Count, Temperature, Duration); the type system enforces dimensional correctness at compile time, so adding grams to milliliters or substituting a volume for a mass is rejected before the program is executed.

A small sample program:

```
function half(n: int) -> int { return n / 2 }

recipe pancakes(servings: int) serves servings {
    ingredient flour : 50 g * servings
    ingredient milk  : 60 ml * servings
    ingredient eggs  : half(servings) * 1 count
    ingredient salt  : 1 pinch

    step "Mix dry" { combine(flour, salt) }
    step "Cook batches" at 180  C  for 3 min {
        repeat servings times { pour(milk) flip() }
    }
}

evaluate scale(pancakes(servings: 4), by: 1.5)
```

Sebesta evaluation criteria (§1.3). Recipix prioritizes **reliability** (strong static typing, dimensional discipline, no shadowing, single-assignment) and **writability** for domain users (concise recipe declarations, implicit Ingredient → Quantity projection in arithmetic). It deprioritizes **cost**: every numeric quantity is normalized to base units at run time, and the dimensional type system is paid for at compile time. **Readability** sits between reliability and writability: mandatory braces on `if/else` (decision #20) and the fixed `at`-before-`for` step-modifier order (decision #23) trade a small writability cost for a grammar that documents the language's design choices.

What makes Recipix a DSL. Dimensional type discipline plus three call-site-only operators (`evaluate`, `scale`, `substitute`) that produce fresh recipe values without mutation. A generic scripting language treats `200` and `500` as raw numbers and lets the programmer combine them into nonsense; Recipix lifts the dimensional structure into the type system, making the nonsense impossible to express.

2. Lexical Structure [P1, revised]

Whitespace separates tokens. Comments start with `//` and run to end-of-line. Identifiers match `[a-zA-Z_][a-zA-Z0-9_]*` (ASCII, case-sensitive) and may not collide with the reserved keyword set. String literals are UTF-8 encoded; any character except `"` and newline is legal inside the quotes; no escape sequences in v1.

Category	Pattern / examples
Identifier	`[a-zA-Z_][a-zA-Z0-9_]*` e.g. `flour`, `pancakes`, `oat_milk`
Integer literal	`[0-9]+` e.g. `0`, `42`, `200`
Float literal	`[0-9]+ "." [0-9]+` e.g. `0.5`, `1.5`, `3.14`
String literal	`"` … (UTF-8, no escapes) `"`
Boolean literal	`true` | `false`
Reserved keyword	`recipe function ingredient step let if else repeat times foreach in at for evaluate scale substitute serves with by ratio return quantity_of`
Type-name keyword	`int float bool Mass Volume Count Temperature Duration Pinch`
Action-verb keyword	`combine mix pour melt whisk blend bake flip add sprinkle drizzle knead`
Unit keyword	`mg g kg ml l tsp tbsp cup °C min hr count pinch`
Operator	`+ - * /  == != < <= > >=  && || !  =`
Separator	`( ) { } [ ] , :  ->`

Two-token quantity literals (decision #7). A quantity such as `200 g` is *two tokens*: an `INT_LIT` or `FLOAT_LIT` followed by a `UNIT_KW`. Only whitespace and `//`-line comments may appear between them. Writing `200g` (no separator) is a lexical error. The two-token form keeps the lexer simple and lets the parser glue `NUMBER UNIT` into a `QuantityLit` AST node at one production. The `°C` token is a single multi-character `UNIT_KW`; the degree symbol may not appear standalone.

3. Syntax [P1, revised]

The complete grammar is summarized below. Productions are unambiguous on the implemented subset; non-trivial design choices visible at the grammar level are annotated.

```
(* Top-level program *)
<program>          ::= { <top_decl> }
<top_decl>         ::= <recipe_decl> | <function_decl> | <eval_stmt>

(* Recipe declaration *)
<recipe_decl>      ::= "recipe" IDENT "(" [ <params> ] ")"
                       "serves" <expr> <block>
<params>           ::= IDENT ":" <type> { "," IDENT ":" <type> }
<block>            ::= "{" { <ingredient_decl> | <step_decl> | <stmt> } "
   }"

(* Function declaration *)
<function_decl>    ::= "function" IDENT "(" [ <params> ] ")"
                       "->" <type>
                       "{" { <stmt> } "return" <expr> "}"
```

```
                        (* error #19: single return at end, enforced by
                           checker *)

(* Ingredient and step declarations *)
<ingredient_decl> ::= "ingredient" IDENT ":" <expr>
<step_decl>       ::= "step" STRING_LIT [ "at" <expr> ] [ "for" <expr> ]
                      "{" { <step_action> } "}"
                        (* decision #23: 'at' before 'for', each at most
                           once *)

(* Statements *)
<stmt>            ::= <let_stmt> | <if_stmt> | <repeat_stmt>
                    | <foreach_stmt> | <action_stmt>
<let_stmt>        ::= "let" IDENT ":" <type> "=" <expr>
<if_stmt>         ::= "if" <expr> <block_stmt> [ "else" <block_stmt> ]
                        (* decision #20: braces mandatory; dangling-else
                           eliminated by construction *)
<block_stmt>      ::= "{" { <stmt> } "}"
<repeat_stmt>     ::= "repeat" <expr> "times" <block_stmt>
<foreach_stmt>    ::= "foreach" IDENT "in" <expr> <block_stmt>

(* Domain-specific top-level operation *)
<eval_stmt>       ::= "evaluate" <expr>

(* Expression precedence ladder - lowest to highest *)
<expr>            ::= <or_expr>
<or_expr>         ::= <and_expr> { "||" <and_expr> }
<and_expr>        ::= <eq_expr>  { "&&" <eq_expr> }
<eq_expr>         ::= <rel_expr> [ ("==" | "!=") <rel_expr> ]
<rel_expr>        ::= <add_expr> [ ("<" | "<=" | ">" | ">=") <add_expr>
   ]
                        (* decision #17: <eq_expr> and <rel_expr> are
                           non-associative -- at most one comparison op
                              per
                           production; chained comparisons are parse
                              errors *)
<add_expr>        ::= <mul_expr> { ("+" | "-") <mul_expr> }
<mul_expr>        ::= <unary>    { ("*" | "/") <unary> }
<unary>           ::= "-" <unary> | "!" <unary>
                    | "quantity_of" "(" <expr> ")"
                    | <primary>
                        (* decision #34: quantity_of is a unary operator,
                           not a function -- dedicated production at the
                           unary level, not routed through <primary> *)

(* Primary expressions and call-site operators *)
<primary>         ::= <int_lit>   [ <unit_kw> ]
                    | <float_lit> [ <unit_kw> ]
                        (* decision #7: quantity literals are TWO tokens
                           *)
                    | <string_lit> | <bool_lit> | IDENT
                    | <list_lit>
                    | <function_call> | <recipe_call>
                    | <scale_call> | <substitute_call>
                    | "(" <expr> ")"
<list_lit>        ::= "[" [ <expr> { "," <expr> } ] "]"
<function_call>   ::= IDENT "(" [ <expr> { "," <expr> } ] ")"
<recipe_call>     ::= IDENT "(" [ <kwarg> { "," <kwarg> } ] ")"
```

```
<kwarg>            ::= IDENT ":" <expr>
<scale_call>       ::= "scale" "(" <expr> "," "by" ":" <expr> ")"
<substitute_call>  ::= "substitute" "(" <expr> ","
                         IDENT ","
                         "with"  ":" IDENT ","
                         "ratio" ":" <expr>
                       ")"
                       (* decision #35: the two ingredient slots are
                          bare IDENT terminals, not <expr>; the grammar
                          makes decision #28 visible at parse level *)

(* Types *)
<type>             ::= "int" | "float" | "bool"
                     | "Mass" | "Volume" | "Count"
                     | "Temperature" | "Duration"
                     | "Pinch"
                       (* decision #32: no user-facing type-parameter
                          syntax; List<T> does not appear in user source
                             *)
```

The grammar is unambiguous on the subset implemented. Operator precedence is encoded by the seven-level expression ladder (`<or_expr>` ↓ `<unary>`); associativity is left for binary arithmetic and logical operators (the `{ ... }` repetition), non-associative for comparisons (the `[ ... ]` optional single operator), and right for unary prefix operators (the recursive `<unary>` in the production).

4. Semantics [P2]

Two non-trivial constructs are given operational semantics in the style of Sebesta §3.5: `scale` (a domain-specific operator) and `foreach` (an iterator loop interacting with scope).

Let S be the state – an environment mapping names to values. Values include `RtQuantity(v, d)` (numeric v in the dimension's base unit and dimension tag d), `RtPinch`, `RtIngredient(name, q)`, `RtRecipe(name, servings, ingredients, steps, step_decls, captured_params, ...)`, `RtList(elems, t)`, and the primitives `int`, `float`, `bool`. We write $\langle e, S\rangle \to v$ for expression evaluation and $\langle s, S\rangle \to S'$ for statement execution.

`scale(r, by: k)` – functional rescaling

```
  <r, S> -> R = RtRecipe(name, n, ingredients, step_decls,
                         captured_params, ...)
  <k, S> -> k : (int | float)
  k > 0
  n' = int(round(n * k))
  for each (id, ing) in ingredients:  ing' = scale_ingredient(ing, k)
  ingredients' = { id -> ing' }
  S_scaled = globals.child()                        (* fresh env *)
  for each (p, v) in captured_params:
      S_scaled[p] = n'   if v was the original servings value
                    v    otherwise
  for each (id, ing') in ingredients': S_scaled[id] = ing'
  for each step_decl in step_decls:
      steps'_i = exec_step(step_decl, S_scaled.child())
  ------------------------------------------------------------------
  <scale(r, by: k), S> ->
      RtRecipe(name, n', ingredients', steps', step_decls,
```

```
                captured_params, ...)

  where:
    scale_ingredient(ing, k) =
      RtIngredient(ing.name, RtQuantity(ing.q.v * k, ing.q.d))
        if ing.q.d in {Mass, Volume, Count}
    scale_ingredient(ing, k) = ing        (* unchanged *)
        if ing.q.d in {Temperature, Duration} or ing.q = RtPinch

    exec_step(step_decl, env) =
      RtStep(description, eval(at_expr, env), eval(for_expr, env),
             accumulate_actions(body, env))
```

Side conditions. If $k \leq 0$ the rule does not apply; the interpreter raises `RuntimeRecipixError`. Banker's rounding (`int(round(·))`) is the rule locked in `part2_plan.md` §2.6 – unbiased on half-way cases, where truncation would bias down and ceiling would bias up. Temperature and Duration remain unchanged because they are intensive quantities in the physical sense: a recipe at `180 °C` does not become `270 °C` when doubled.

Step bodies are re-executed under the scaled `servings` binding. Recipe instantiation captures the original parameter map (`captured_params`) and the unevaluated step declarations (`step_decls`) on the `RtRecipe` value. When `scale` runs, it builds a fresh evaluation environment in which the `servings` parameter is rebound to n' (and all ingredient bindings are rebound to the scaled values), then re-executes each step declaration's body against that environment. The visible consequence in sample 1 is that `scale(pancakes(servings: 4), by: 1.5)` renders "serves 6" with *six* `pour/flip` cycles in the step body – `repeat servings times` rebinds to 6 in the scaled evaluation, not 4. This matches the user-intuitive reading of `scale`: doubling a recipe doubles the work, not just the header. Spec §9 ("multiplies the servings field by the scalar") is silent on loop bodies, but re-execution is the consistent reading once `servings` is the controlling parameter for any step-body iteration referring to it. Referential transparency (decision #25) is preserved because `scale` still produces a fresh `RtRecipe` value without mutating the original – the re-execution happens entirely inside the new value's construction.

`foreach x in <list> { body }` – homogeneous iteration

```
  <list_expr, S> -> RtList(elems, T)
  S_0 = S
  for i in [0, |elems|):
      S'_i = S_i[x -> elems[i]]          (* bind loop var in fresh scope
          *)
      <body, S'_i> -> S''_{i+1}
      S_{i+1} = S''_{i+1} \ {x}          (* loop var goes out of scope *)
  ------------------------------------------------------------------
  <foreach x in list_expr { body }, S> -> S_{|elems|}
```

Side conditions. The list value must have $|elems| \geq 0$ (the empty list produces zero iterations). The loop variable x is bound to type T in each iteration's fresh scope; the binding goes out of scope at the iteration's end. x may not shadow a name visible in S (decision #12, enforced by the type checker before this rule fires). The list expression is evaluated *once*, before any iteration – consistent with decision #18 (operand evaluation order is left-to-right and fully defined). Within a single iteration the loop variable is immutable (decision #13); between iterations it is rebound, which is the only rebinding form in Recipix v1.

Design alternative. A `while` form would have worked, but `foreach` pairs naturally with Recipix's no-mutation discipline: the homogeneous-list invariant is carried in the loop variable's

type, and the loop is intrinsically bounded by the list's size. A `while` whose condition can change between iterations only makes sense in a language with side effects.

5. Type System [P2]

Primitive types and value sets. Recipix has six families of primitive types:

- `int` – 64-bit signed integers, value set $[-2^{63}, 2^{63}-1]$.
- `float` – IEEE 754 doubles.
- `bool` – {`true`, `false`}.
- **Quantity types** (Sebesta §6.2 terminology: *abstract data types representing physical dimensions*): `Mass`, `Volume`, `Count`, `Temperature`, `Duration`. Internally each is a base-unit numeric value plus a dimension tag; values are normalized to base units (`g`, `ml`, `count`, `°C`, `min`) at run time.
- `Pinch` – a separate primitive (decision #4). Constructed only by the literal `1 pinch`; admits no arithmetic, no comparison, no scaling, no substitution-by-ratio. The integer prefix is required syntactically but semantically ignored: `1 pinch` and `5 pinch` denote the same value.

Strongly typed (Sebesta §6.12). Yes – strictly. Every operator is defined only on matching operand types; every implicit coercion is listed below; every type error is caught at compile time (with two exceptions noted at the end of this section). The dimensional discipline adds a stronger guarantee than a generic strongly-typed language: type identity is not only "these have compatible representations" but also "these denote the same physical dimension."

Implicit coercion (Sebesta §7.4). Two coercions are allowed.

- **Within-dimension unit coercion.** Two quantities of the same dimension expressed in different units are interchangeable: `200 g + 1 kg` evaluates to `1200 g`. Decision #9 treats this as a representation-level conversion, not a type-level coercion – the dimension is preserved.
- **`int` widens to `float`** in mixed-mode arithmetic. `1 + 2.0` yields `3.0` of type `float`; `int` is information-preserving in this direction (every 64-bit int $\leq 2^{53}$ is representable as a double).

The *narrowing* direction is *never* implicit: `float` → `int` requires an explicit conversion (reserved for a v2 `to_int` builtin). Cross-dimension coercion is rejected at compile time: there is no implicit conversion from `Mass` to `Volume`, even when the user provides a density.

Type equivalence (Sebesta §6.14). A deliberate split (decision #10):

- **Structural** for `Quantity<D>`, `Ingredient`, and `List<T>`. Two `Mass` values are the same type regardless of unit; two `Ingredient` records with the same field shape are the same type; two lists with the same inferred element type are the same type. The reasoning: these types model *data shape* – the field set carries meaning, and the closed v1 type-name set forecloses the "hypothetical user-defined record with the same shape" concern that motivates name equivalence in open systems.
- **Name** for `Recipe`. A recipe's identity comes from its declaration name, not from its ingredient or step shape. `pancakes` and `crepes` that happen to share `{flour, milk, eggs}` are *not* interchangeable in `scale` or `substitute`; accidental shape collisions at the top of the type lattice would be a worse failure mode than the ergonomic cost of the strict rule.

Structured-type design decisions (Sebesta §6.5–6.7). The structured type that bears the full design weight is the **list** (decision #5, decision #32). Following Sebesta §6.5:

- **Element type:** `List<T>` is homogeneous – every element must have the same type T. Mixed-dimension lists are a compile-time error (spec §10 error #3).
- **Index range:** not user-facing. Recipix v1 has no `list[i]` indexing operator; `foreach` is the only traversal form. The grammar does not admit subscripting.
- **Subscript-check semantics:** not applicable for the same reason. An empty-list `foreach` produces zero iterations with no error.
- **Static vs. dynamic:** list type is *inferred* at literal construction and at `foreach` loop entry – users do not write `List<T>` annotations in source code (decision #32). Lists are dynamic in length but static in element type.

The `Ingredient` record (decision #5) is a two-field structural type with `name : string-label` and `quantity : Quantity<D> | Pinch`. Its identity in program semantics is the symbol-table binding (decision #28), not the `name` field's contents – which is what makes `substitute(r, flour, with: oat_milk, ratio: 1.0)` a binding-resolution operation rather than a string match. The bare-IDENT restriction on the substitute slots (decision #35) makes this visible at the grammar level: the slots are `IDENT` terminals, not `<expr>`, so the parser refuses `substitute(r, flour + salt, with: ...)` outright.

Two exceptions to "catches every type error at compile time." (a) Negative quantities – the unary `-` operator syntactically applies to quantities and the interpreter accepts negative quantity values, but the type checker does not flag them in v1 (spec §3, line on unary minus). (b) Inside an empty list literal `[]` the inferred element type is `List<?>`, which the checker handles defensively but does not yet integrate with the `foreach` loop-type inference. Both are deliberately scoped out of v1 and named in the D5 retrospective.

6. Names, Binding, Scope, Lifetime [P1, revised]

Legal identifiers. Match `[a-zA-Z_][a-zA-Z0-9_]*`, ASCII only, case-sensitive. Identifiers must not collide with reserved keywords, type-name keywords, action-verb keywords, or unit keywords. There are no hyphens in identifiers (the lexer treats `-` as the arithmetic operator).

Bindings: compile time vs. run time (Sebesta §5).

Binding	Compile time	Run time
Recipe declarations (name and signature)		
Function declarations (name and signature)		
Ingredient *types* (dimensions)		
Ingredient *quantity values*		
Recipe parameters		
Function parameters		
`foreach` loop variables (type)		
`foreach` loop variables (value)		
`let`-bound names (type, via annotation)		
`let`-bound names (value)		

Scoping rule: static (lexical) – decision #11. Each of the following constructs opens a fresh scope: a `recipe` body (with its parameters and ingredients in scope), a `function` body (with its parameters in scope), a `step` body (with the recipe's ingredients in scope), an `if` and `else` block (each independently), a `repeat` body, and a `foreach` body (with the loop variable in scope). Inner scopes can read outer-scope names. Parameter lists live in the *same* scope as the body they introduce (plan §2.5 scope convention): a `let x` inside `function f(x: int) { ...}` is a shadowing error, not a fresh binding in a nested scope. The name lookup walks the parent-pointer chain in the `Environment` class – if the user asks for `flour` in a step body, the lookup starts in the step body's scope, walks up into the recipe body's scope, and resolves at the ingredient declaration.

What would break if we switched to dynamic scoping? Two concrete things, both ugly. (1) `quantity_of(flour)` resolves through the enclosing recipe's symbol table; under dynamic scoping it would resolve through whichever scope happened to be active at run time, breaking the type rule that `quantity_of` returns the dimension of the operand's quantity field – there would be no single "operand" the type checker can reason about. (2) The `Substitute` call's bare-IDENT slot resolution (decision #35) becomes ambiguous: dynamic scoping would let `substitute(r, milk, ...)` resolve `milk` against the caller's scope rather than the recipe's, which contradicts the binding-identity rule of decision #28.

No shadowing (decision #12) and single-assignment (decision #13). Declaring an identifier already visible in any enclosing scope is a compile-time error – the type checker enforces this via `Environment.declare_check()`. Within their scope, `let`-bound names, ingredient names, and recipe/function parameters are immutable. Loop variables (`foreach x in ...`) are rebound on each iteration; within a single iteration they are immutable. There is no assignment statement in Recipix v1 (decision #27 – `let` is the only binding form).

Lifetime (Sebesta §5.4).

Entity	Lifetime
Recipe / function declarations	Static (whole program)
Recipe parameters	Stack-dynamic, per instantiation
Function parameters	Stack-dynamic, per call
Ingredients within a recipe	Stack-dynamic, per recipe instantiation
`let`-bound names	Stack-dynamic, per enclosing scope
`foreach` loop variables	Stack-dynamic, per iteration

No explicit-heap-dynamic lifetimes exist in v1 – there is no `new`, no allocation operator, no garbage-collected handle exposed to the user.

7. Expressions & Assignment [P2]

Operator precedence and associativity (Sebesta §7.2).

Level	Operators	Associativity
1	unary `-`, `!`, `quantity_of(...)`	right
2	`*`, `/`	left
3	`+`, `-` (binary)	left
4	`<`, `<=`, `>`, `>=`	**non-associative**
5	`==`, `!=`	**non-associative**
6	`&&`	left
7	`||`	left

Parenthesization overrides precedence. The non-associative comparison levels are enforced at parse time: `a < b < c` is a parse error, not a type error. This catches a category of bug – chained comparisons that silently parse as `(a < b) < c` and try to compare a `bool` against the original right operand's type – at the earliest possible phase. The trade-off is that users wanting transitive chaining must write `a < b && b < c` explicitly.

Short-circuit evaluation (Sebesta §7.5). `&&` and `||` short-circuit left-to-right. The interpreter evaluates the left operand first; if its value determines the result (`false` for `&&`, `true` for `||`), the right operand is not evaluated. Short-circuit is observable only in the sense that the right operand is not type-checked at run time in skipped paths – but the type checker has already proven both operands are `bool`, so short-circuit cannot hide a type error.

Operand evaluation order (Sebesta §7.6). Left-to-right, fully defined (decision #18). Stronger than C's "unspecified" rule, weaker than "nothing could be different" (because in the absence of side effects the order is unobservable). Recipix has no mutation in v1, so the choice is unobservable to the user, but locking the order matters for error reporting: in `let x : int = f(a/0) + g(b/0)`, the division-by-zero from `a/0` is the one that fires, not `b/0`. Users debugging a run-time error get a predictable line number.

Assignment is a statement, not an expression (decision #27). The only binding form is `let <name> : <type> = <expr>`, which is a statement. There is no assignment-in-expression form. The trade-off is mild verbosity in some patterns (`if (x = compute()) > 0` is not legal in any expression position), but the gain is that no expression has a side effect – which lets decision #18's left-to-right evaluation order be unobservable and lets the entire language be referentially transparent in v1. Recipix has no compound assignment (`+=`, `*=`), no increment (`++`), and no augmented assignment of any kind.

8. Design Rationale [P2]

One paragraph per non-trivial design decision, in each author's own voice. The seven decisions explicitly identified by the rubric – coercion, type equivalence, structured-type design, precedence, short-circuit, operand order, and assignment-as-statement – are covered below.

Coercion (Sebesta §7.4) – İsmail. Recipix's coercion rules are deliberately asymmetric. Within-dimension unit coercion is loss-free at the type-system level – `200 g` and `1 kg` denote the same dimension, just under different representations – so the language treats this as an internal numeric conversion rather than a type-system event. `int` → `float` is the other allowed direction because every 64-bit int small enough to matter for cooking is representable in IEEE 754 double precision without loss. `float` → `int` is *never* implicit: Sebesta §7.4 explicitly flags narrowing coercions as the dangerous kind, and silent truncation in scaling (`int(round(servings * 1.5))` vs. `int(servings * 1.5)`) is exactly the bug a user would have a hard time debugging. The

trade-off accepted: explicit conversions in v1 are slightly noisier than C-style implicit narrowing, but they remove a whole class of silent-data-loss errors.

Type equivalence (Sebesta §6.14) – İsmail. The split between structural (for `Quantity<D>`, `Ingredient`, `List<T>`) and name (for `Recipe`) is the most contested design call in the spec. Sebesta §6.14 frames the choice as one rule per language; Recipix takes both – because the records modeling *data shape* (Ingredient, List, Quantity) want structural so they interoperate across scopes, while the type modeling *identity* (Recipe) wants name so accidental shape-matches cannot silently substitute one recipe for another in `scale` or `substitute`. The trade-off: a Recipix user cannot define a custom record type that *happens* to be "equivalent to an Ingredient," but in v1 there is no user-defined record syntax anyway (decision #32), so the restriction costs nothing.

Structured-type design (Sebesta §6.5–6.7) – İsmail. The list is the only user-facing structured type in Recipix v1, and it is intentionally minimal. Element type is fixed at literal construction (homogeneous lists, decision #6); type is inferred (no user-written `List<T>`); no indexing (`foreach` is the only traversal form); the empty list has a defined "no iterations" behavior. This matches Sebesta §6.5's framing of lists as "sequences whose length is dynamic but whose element type is fixed." What is *not* in v1: arbitrary nested type parameters in user code (no `List<List<Mass>>` annotation), variant types, sum types, and recursive type definitions. Recipix v1 deliberately stops at the single-level homogeneous list because that is the minimum sufficient for the language's domain – step bodies iterating over ingredient collections, scaling iterating over fixed-arity recipe parts.

Operator precedence and non-associative comparison – Berk. The hardest call here was whether to make comparisons non-associative or left-associative with the standard `(a < b) < c` chaining semantics. Left-associative is the C/Java/Python-without-chaining default and would have made the precedence ladder simpler – one level for all six comparison ops instead of two non-associative levels. I picked non-associative because of the failure mode left-associative chaining produces: `1 kg < flour < 2 kg` parses cleanly under C semantics, then evaluates `1 kg < flour` to a `bool`, then tries to compare that `bool` against `2 kg` and gets a type error. The user reads the program, sees a sensible-looking transitive comparison, and gets a confusing type-error message at run time. Splitting comparisons into `<rel_expr>` and `<eq_expr>` and making both non-associative catches this at parse time with a clean "chained comparison not allowed" message. The trade-off accepted: users who want transitive chaining write `(1 kg < flour) && (flour < 2 kg)` – a small writability cost paid in the currency of reliability and clearer error messages.

Short-circuit `&&` and `||` – Berk. Short-circuit is the standard choice and the one Sebesta §7.5 frames as the user-friendly default – it is what programmers expect, and the alternative (eager evaluation of both operands) would force users to write `if x != null && x.field > 0` as nested `if`s with awkward indentation. The interesting design question was *which direction* to short-circuit. Left-to-right matches the operand-evaluation rule from decision #18 and gives users one mental model for both: expressions are evaluated in the order they are written, and logical operators stop early when they have the answer. Right-to-left short-circuiting would have been internally consistent if the operand-evaluation rule were also right-to-left, but Sebesta §7.6 notes that left-to-right is the strongly dominant convention.

Operand evaluation order is fully defined left-to-right – Berk. Decision #18 over-commits compared to most language specs, which leave operand order "unspecified" so compilers can reorder for optimization (C, C++). The C-style choice has a real cost in pedagogical

languages and in DSLs: a student running the same program twice can get different run-time errors because two division-by-zero candidates fire in different orders on different platforms. For Recipix this would be especially confusing because the error messages cite line numbers, and a non-deterministic line number breaks the reliability promise the language makes. I locked left-to-right and accepted the cost (the interpreter cannot reorder operands for any optimization) because reliability and predictable error reporting matter more than performance in v1 – and "performance" was already deprioritized in spec §1's evaluation-criteria framing.

Assignment is a statement, not an expression – Berk. Decision #27 was a deliberate import from functional-language discipline: with `let` as the only binding form and assignment as a statement, every expression in Recipix is referentially transparent – the same expression evaluated in the same scope produces the same value, every time. The trade-off is that some patterns require slightly more code (no `while ((line = read()) != null)` form). I accepted this because (a) v1 has no mutation and no I/O loop, so the missing pattern does not arise in real Recipix programs, and (b) it lets decision #18's left-to-right evaluation order be entirely unobservable, which simplifies the operational semantics in §4 above – every rule for binary operators can ignore the question "what if the left operand modified something the right operand reads?" because no operand can modify anything. The cost is a v1-only restriction; if Recipix v2 introduces mutation, assignment-as-statement is the first rule that should be revisited.

End of D1 [P2]. Companion documents: D3 (test report and sample outputs), the locked language specification at `recipix_v4_1_spec.md`, and the joint contract at `part2_plan.md`.